香山 Kunminghu V2 自定义 vdot.vv 指令设计与验证

摘要：本项目面向 CIE 全国 RISC-V 高水平创新及应用大赛香山社区 RISC-V 应用创新赛道，基于 XiangShan Kunminghu V2 处理器实现自定义向量点积指令 vdot.vv。指令面向 VLEN=128、SEW=8 的向量状态，仅对两个源向量的低 8 个 signed INT8 元素执行乘加归约，将 signed INT32 结果写入目的向量低 32 位并清零高 96 位。项目完成了 Phase 1 环境部署与标准 RVV 基线验证、Phase 2 RTL 指令实现与波形验证，以及 Phase 3 NEMU 参考模型、RTL-NEMU 协同仿真、数学等价 RVV baseline 和受控性能评估。六类合法语义输入、非 e8 非法指令行为、标准 RVV 回归和等价 baseline 均通过验证。正式性能测试显示，compute-only 场景的稳态计算吞吐/周期加速比为 4.769517 倍，包含配置、加载、计算和存储的 end-to-end 周期加速比为 1.432995 倍。

关键词：Kunminghu V2；向量扩展；自定义指令；vdot.vv；DiffTest

一、项目概述

1.1 项目目标

项目目标是在香山 Kunminghu V2 处理器中增加一条可验证、可复现的向量点积指令，并形成从指令编码、译码、执行单元、仿真参考模型到性能评估的完整技术闭环。实现过程中保持既有 RVV 指令行为不变，使用标准 RVV 算法构造数学等价 baseline，并通过 RTL 与 NEMU 的一致性验证支撑最终结论。

1.2 系统组成

项目由 XiangShan RTL、NEMU reference model、nexus-am workload、标准 RVV 回归程序和性能 benchmark 组成。Phase 1 建立可运行的处理器仿真与 DiffTest 环境；Phase 2 在后端译码中接入 vdot.vv，并在 VIMacU 封装内新增独立的 VDOT 乘法阵列与结果旁路；Phase 3 将同一语义同步到 NEMU，并以相同测试向量验证 custom 指令和标准 RVV baseline。

表 1 阶段内容以及成果

阶段	主要内容	最终状态
Phase 1	搭建 XiangShan/NEMU 仿真与 DiffTest 环境；运行 Hello 程序；验证标准 RVV vadd.vv 波形	PASS
Phase 2	完成 vdot.vv RTL 接入；开展边界及正负数测试；检查执行波形	PASS
Phase 3.1	扩展 NEMU 参考模型；完成 RTL-NEMU DiffTest 一致性验证	PASS
Phase 3.2	构建数学等价的标准 RVV baseline，并完成对照验证	PASS
Phase 3.3	基于受控 rdcycle 采样开展性能评估	PASS

1.3 技术路线

技术路线遵循“先建立可运行基线，再实现 custom 指令，最后完成协同验证与性能对比”的顺序。每一阶段保留源码、运行日志、反汇编、波形截图和 SHA256 校验信息，正式报告使用最终封存证据中的结果，不用临时重跑结果替代。

二、Phase 1 环境与标准 RVV 基线

2.1 仿真环境验证

Phase 1 首先在 XiangShan RTL simulator 中运行 Hello workload，并启用 NEMU reference model 和 DiffTest。运行日志显示参考模型正常加载，程序输出 Hello 信息，最终在目标 PC 命中 HIT GOOD TRAP，说明处理器仿真、内存映射、参考模型连接和基本提交路径均可用。

The first instruction of core 0 has commited. Difftest enabled.

hello xiangshan, I am 林淮青

Core 0: HIT GOOD TRAP at pc = 0x8000014c

2.2 标准 RVV vadd.vv 回归

在 custom 指令实现前，项目使用标准 RVV vadd.vv 完成向量回归。输入 A=[1,2,3,4] 、 B=[10,20,30,40] ， 输 出 C=[11,22,33,44] ， 运 行 日 志 记 录 VECTOR_ADD_PASS 和 HIT GOOD TRAP。该回归在后续 Phase 2 与 Phase 3 中持续作为既有 RVV 行为的保护性检查。

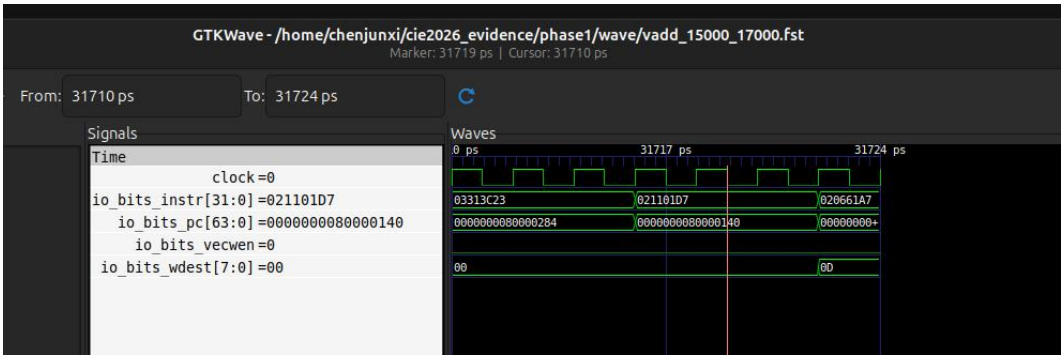


图 1 标准 RVV vadd.vv 波形证据

vsetvli t0,a3,e32,m1,ta,ma

vle32.v v1,(a0)

vle32.v v2,(a1)

vadd.vv v3,v1,v2

vse32.v v3,(a2)

三、vdot.vv 指令设计

3.1 指令语义

vdot.vv 的冻结语义为：在 VLEN=128、SEW=8 条件下，读取 vs1 和 vs2 的低 8 个字节，将每个字节解释为 signed INT8，执行 8 项有符号乘法并累加为 signed INT32。结果的 32 位位模式写入 vd[31:0]，vd[127:32] 全部清零。源向量字节 8 至 15 不参与计算，计算项数量固定为 8，不随 vl 改变。

sum = Σ(i=0..7) signed_int8(vs1.byte[i]) × signed_int8(vs2.byte[i])

vd[31:0] = uint32(sum)

vd[127:32] = 0

3.2 编码与译码

指令使用 custom-0 opcode 0x0b，bit[25] 固定为 1，属于无掩码形式。已验证的实例为 vdot.vv v10,v8,v9，对应机器码 0x0294050b，指令地址为 0x00000000800001ac。译码阶段要求 SEW=e8；当 SEW 不等于 8 时，冻结 RTL 产生 illegal instruction。

表 2 字段以及说明

字段	值	说明
opcode	0x0b	custom-0
vd	v10	目的向量寄存器
vs1	v8	第一个源向量寄存器
vs2	v9	第二个源向量寄存器
SEW	8	唯一合法元素宽度
fuOpType	0x0e0	VIMac 内部操作类型

3.3 RTL 执行通路

RTL 修改集中在 VDotType.scala、VecDecoder.scala 和 VIMacU.scala。译码器将 custom-0 指令映射到 VDOT fuOpType；VIMacU 在既有向量功能单元封装内新增独立的 VDOT 数据通路，从两个源向量取低 8 个 signed INT8 元素，形成 8 个并行乘积并通过平衡分层加法树归约。结果经过独立结果寄存器延迟后，在输出端由 VDOT 旁路选择，绕过常规逐元素 MGU 结果，写入 128 位结果总线的低 32 位，高 96 位清零。该实现复用了 VIMacU 的控制、流水封装和结果时序接口，但 VDOT 的乘法阵列与结果选择路径是独立的。RTL 证据显示该功能单元 latency 为 2 cycles，这一微架构属性与后文 benchmark 的稳态吞吐加速比分开报告。

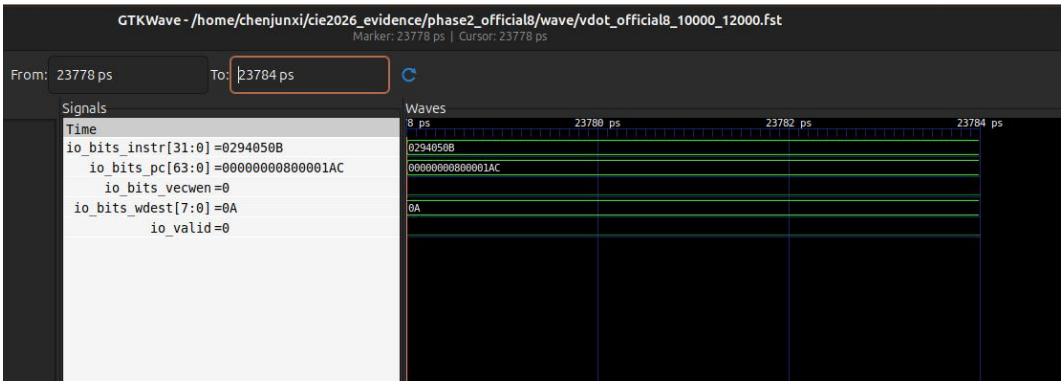


图 2　vdot.vv 指令定位波形

四、Phase 2 RTL 验证

4.1 功能测试矩阵

Phase 2 对正数、负数、最大正值、最大负值、混合符号和高半区 poison 六类输入进行了验证。测试既覆盖算术边界，也覆盖源向量高 8 字节必须被忽略的结构性约束。

表 3 测试类型以及期望结果

测试类型	期望结果	验证含义
Positive	36	基本有符号乘加
Negative	-36	负数累加与结果编码
Max positive	131072	8×(-128×-128)
Max negative	-130048	8×(-128×127)
Mixed sign	127	混合符号输入
Upper-half poison	127	高 8 字节不影响结果

4.2 波形证据

最大正值测试的波形在 23778 ps 定位到目标指令，在 23844 ps 观察到 VIMac 输入有效，fuOpType=0x0e0；23846 ps 内部结果为 0x00020000，23848 ps 输出有效并保持该结果。该值等于 8×(-128×-128)=131072，与低 8 个 signed INT8 元素的理论计算一致。

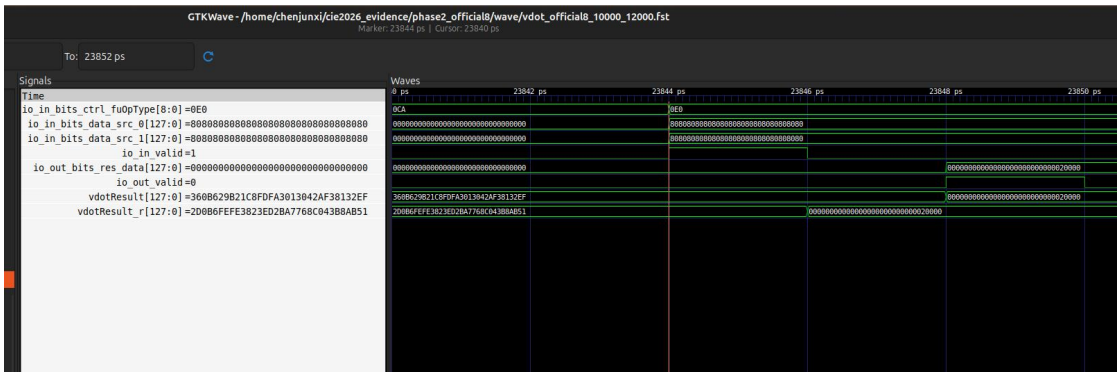


图 3　VIMac 输入与 VDOT 操作类型

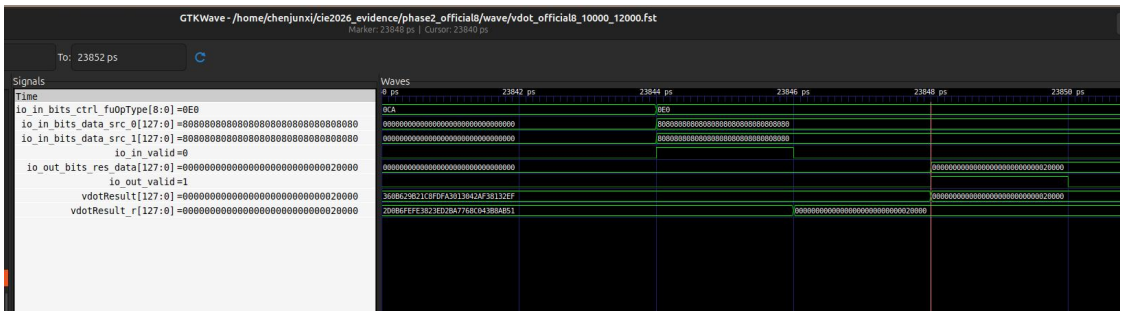


图 4　VDOT 输出结果与高位清零

4.3 标准 RVV 回归保持

Phase 2 同时保留标准 RVV vadd.vv 回归。日志中出现 Difftest enabled、VECTOR_ADD_PASS 和 HIT GOOD TRAP，说明新增 VDOT 译码与执行路径未破坏已有标准向量指令。Phase 2 的 custom VDOT 功能测试使用 --no-diff；NEMU 参考模型协同验证在 Phase 3 单独完成。

五、Phase 3 协同仿真与参考模型

5.1 NEMU 参考模型

Phase 3 在 NEMU 中增加 custom-0 vdot.vv 的解码和执行语义，涉及 isa-all-instr.h、decode.c 和 rvv/vcompute.h。参考模型限定 SEW=e8，按已冻结 RTL 的低 8 字节、signed INT8、signed INT32 归约和高位清零规则更新向量目的寄存器，并同步更新 vector dirty 状态。最终参考模型 commit 为 55c09a3669d52ae6f6a809bab6e594939b82e512。

5.2 RTL-NEMU DiffTest 结果

六类合法 vdot.vv 输入均在真实 XiangShan RTL 与最终 NEMU reference model 中通过 DiffTest，并以 GOOD TRAP 结束。另行完成 SEW 非 e8 的 illegal-instruction 测试和标准 RVV vadd.vv 回归，确认 custom 指令语义、异常边界和既有 RVV 行为均满足要求。

表 4 类别以及对应的结果

类别	结果	状态
Positive	+36	PASS
Negative	-36	PASS
Max positive	+131072	PASS
Max negative	-130048	PASS
Mixed	127	PASS
Upper-half poison	127	PASS
SEW != 8	illegal instruction	PASS
RVV vadd.vv	回归一致	PASS

5.3 验证结论

协同仿真证明了 NEMU reference model 与冻结 RTL 在正式测试域内的行为一致性。验证重点不是只比较最终算术数值，还检查了操作数索引、SEW 合法性、目的寄存器高位清零以及高半区输入忽略等架构约束。

六、数学等价标准 RVV baseline

6.1 baseline 指令序列

为了得到公平的性能参照，项目构造了完全不使用 custom VDOT 的标准 RVV baseline。该路径先以 e8,mf2 设置 VL=8，加载两个源向量，再使用 widening multiply 生成 INT16 乘积，最后以 widening reduction 得到 32 位有符号和。

```
vsetivli zero,8,e8,mf2,ta,ma
vle8.v          v8,(a0)
```

```
vle8.v          v9,(a1)
vwmul.vv        v10,v8,v9
vsetivli        zero,8,e16,m1,ta,ma
vmv.v.i         v12,0
vwredsum.vs v14,v10,v12
vsetivli        zero,1,e32,m1,ta,ma
vse32.v         v14,(a2)
```

6.2 等价性依据

Kunminghu V2 的 VLEN=128；在 SEW=8、LMUL=mf2 时，VLMAX=8，因此 baseline 恰好处理低 8 个元素。vwmul.vv 将 signed INT8 乘积 widening 为 INT16，避免 e8 中间结果溢出；vmv.v.i 提供零初值；vwredsum.vs 完成 signed widening reduction 并得到 32 位结果。baseline 反汇编中 custom VDOT 机器码计数为 0。

6.3 baseline DiffTest

标准 RVV baseline 使用与 custom VDOT 相同的六类语义输入，全部得到一致结果，并通过 RTL-NEMU DiffTest。该结果把功能等价性和性能差异分离开来：两条路径计算相同的数学对象，区别仅在于 custom 指令将核心计算压缩为单条指令。

七、性能评估

7.1 测量方法

性能测试在 XiangShan RTL 上使用 guest rdcycle 作为主指标，采用 --no-diff 运行。功能正确性已由前述 DiffTest 独立证明，因此周期测量不把 NEMU reference model 的执行计数混入 DUT 微架构周期。每条路径先执行 16 次预热，每个样本静态展开 256 次操作，共采集 5 个样本；测量区内无循环分支和 printf，custom 与 baseline 使用相同输入和 counter boundary。

表 5 测量范围以及包含内容

测量范围	包含内容	用途
Compute-only	源向量已加载，仅比较核心计算	衡量稳态计算吞吐
End-to-end	配置、两次 load、计算、store	衡量完整 kernel 成本

7.2 Compute-only 结果

Compute-only 测量中，custom VDOT 每次 dot product 使用 1 条指令，标准 RVV baseline 使用 5 条核心计算指令。custom 中位数为 269 cycles/256 ops，baseline 中位数为 1283 cycles/256 ops，对应每次操作 1.050781 与 5.011719 cycles。

表 6 指标以及对应结论

指标	**Custom VDOT**	**Standard RVV**	结论
Median cycles / 256 ops	269	1283	custom 更低
Cycles / op	1.050781	5.011719	4.769517x
Static instructions / op	1	5	5.000000x

7.3 End-to-end 结果

End-to-end 测量把向量配置、两次源向量加载、点积计算和结果存储全部纳入。custom 中位数为 1776 cycles/256 ops，baseline 中位数为 2545 cycles/256 ops，对应每次操作 6.937500 与 9.941406 cycles，周期加速比为 1.432995x。

表 7 指标以及结论

指标	**Custom VDOT**	**Standard RVV**	结论
Median cycles / 256 ops	1776	2545	custom 更低

指标	Custom VDOT	Standard RVV	结论
Cycles / op	6.937500	9.941406	1.432995x
Static instructions / op	6	9	1.500000x

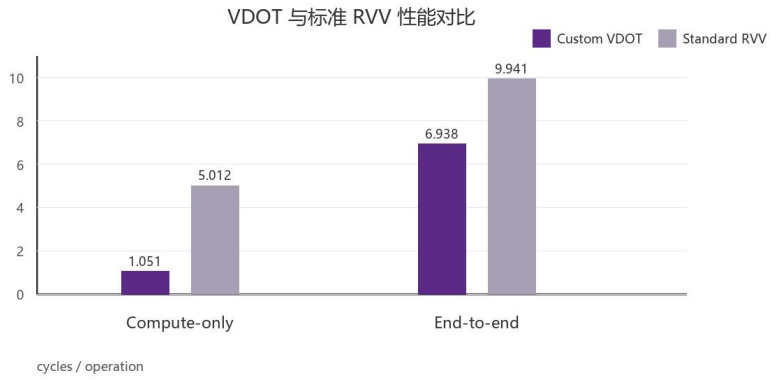


图 5　Custom VDOT 与标准 RVV cycles/op 对比

7.4 稳定性与指标解释

正式五组样本中，custom compute 为 268 或 269 cycles，baseline compute 五次均为 1283 cycles；两条路径的 end-to-end 结果分别稳定为 1776 和 2545 cycles。固定 counter boundary 开销保留在原始结果中，不进行未观测的 overhead 扣除，256 次展开操作用于摊薄固定边界开销。

报告中的 4.769517x 是 256 次静态展开 dot-product 流的稳态 compute throughput/cycle speedup，不是单条 VDOT 指令的 latency speedup。VDOT 功能单元 latency 为独立的 2 cycles 设计属性；end-to-end 结果则反映配置、访存和存储共同参与后的完整 kernel 成本。

rdinstret 仅作为辅助 CSR 观察，精确静态指令数以冻结 benchmark 的 objdump 为准。whole-program cycleCnt、Guest cycle spent 和 Host time spent 含启动、打印、控制流、陷阱与模拟器开销，不能作为本项目的硬件加速比。

表 8 Compute-only 原始周期样本

样本	VDOT 周期	RVV 周期	加速比
样本 0	269	1283	4.769517x
样本 1	269	1283	4.769517x

样本	VDOT 周期	RVV 周期	加速比
样本 2	268	1283	4.787313x
样本 3	269	1283	4.769517x
样本 4	268	1283	4.787313x

表 9 End-to-end 原始周期样本

样本	VDOT 周期	RVV 周期	加速比
样本 0	1776	2545	1.432995x
样本 1	1776	2545	1.432995x
样本 2	1776	2545	1.432995x
样本 3	1776	2545	1.432995x
样本 4	1776	2545	1.432995x

八、项目总结

8.1 完成情况

本项目已完成从 XiangShan/NEMU 基线环境、标准 RVV 回归、自定义 vdot.vv RTL 实现、波形定位、NEMU 参考模型、协同仿真、数学等价 baseline 到受控性能评估的完整闭环。技术证据已在最终提交目录中封存，后续工作主要是正式报告、演示材料和提交打包，不需要重新编译 XiangShan 或 NEMU。

8.2 主要结论

1. vdot.vv 在冻结语义下只计算低 8 个 signed INT8 元素，并将 signed INT32 结果写入低 32 位，高 96 位清零。

2. 六类合法输入、SEW 非法边界和标准 RVV vadd.vv 回归均通过 RTL-NEMU 或既有 DiffTest 验证。

3. 标准 RVV baseline 与 custom VDOT 在冻结测试域内数学等价，baseline 不包含 custom VDOT 编码。

4. compute-only 稳态计算吞吐/周期加速比为 4.769517x，end-to-end 周期加速比为 1.432995x。

5. VDOT FU latency 为 2 cycles，与 benchmark 的稳态 throughput 指标分

属不同层次。

附录

证据类别	文件位置	用途
Phase 1 Hello	01_phase1/hello_run_summary.txt	环境与 DiffTest 基线
Phase 1 RVV	01_phase1/vadd_run_summary.txt	标准 vadd.vv 回归
Phase 2 设计	02_phase2/Phase2_VDOT_Design_and_Verification.txt	RTL 语义与测试
Phase 2 波形	02_phase2/wave/	指令、输入、输出定位
Phase 3 合同	03_phase3/PHASE3_VDOT_CONTRACT.md	冻结架构语义
Phase 3 协同仿真	03_phase3/cosim/STEP5V_LEGAL_VDOT_NEMU_V2_SUMMARY.txt	合法语义结果
RVV baseline	03_phase3/baseline/STEP6G_RVV_BASELINE_DIFFTEST_FIXED.log	等价性验证
性能摘要	03_phase3/perf/STEP7D_PERFORMANCE_SUMMARY.txt	正式 cycles/op 与 speedup
性能解释	03_phase3/perf/STEP7E_PHASE3_PERFORMANCE_FINAL_INTERPRETATION.txt	指标边界与解读